Laboratory Work No. 4. “Data Layer in a FastAPI Application for Image Processing and Pattern Recognition”

Table of Contents

Laboratory Work No. 4. “Data Layer in a FastAPI Application for Image Processing and Pattern Recognition”

1. Purpose of the Laboratory Work

Master the creation of a persistent data layer in a multi-tier FastAPI web application. Learn how to store images in a relational SQLite database using the DB-API standard (PEP 249). Become familiar with the Pydantic model `Picture`, which contains a `numpy.ndarray` object, and the integration of the OpenCV library at the service layer for preliminary image processing. Acquire skills in performing CRUD operations on binary data (BLOB).

2. Overview and Theoretical Foundations

In previous laboratory works, images were stored only in RAM. This approach is convenient for prototyping but is unsuitable for real applications — all data is lost after the server is restarted.

In this work we create a **persistent data layer** — an SQLite database that reliably stores images even after the computer is turned off. We strictly follow the principles described in Chapter 10 of Bill

Lubanovich's book "FastAPI. Web Development with Python" (2024): we use a three-tier application architecture:

- **web/** — presentation layer (FastAPI routers, HTTP endpoints, request validation).
- **service/** — business logic layer (image processing with OpenCV and calls to the data layer occur here).
- **data/** — data layer (direct interaction with SQLite via DB-API).
- **model/** — Pydantic models for validation and serialization of data.

This separation allows: - Easy independent testing of each layer.

- Changing the database (for example, switching to PostgreSQL) without modifying the code of the web and service layers.

- Scaling the application for AI tasks (classification, segmentation, object detection, etc.).

3. Project Structure

Create a new project and organize the folders **exactly as shown** (all folders are at the same level as the project root):

```
fastapi-image-processing/
├── data/                  # Data layer (SQLite + DB-API)
│   ├── __init__.py
│   ├── init.py
│   ├── pictures.py
├── db/                    # Folder for the database file
├── model/                 # Pydantic models
│   ├── __init__.py
│   └── pictures.py
├── service/               # Business logic + OpenCV
│   ├── __init__.py
│   └── pictures.py
├── src/                   # Application entry point
│   └── main.py
├── web/                   # FastAPI routers
│   ├── __init__.py
│   └── pictures.py
├── requirements.txt
├── .env
└── README.md
```

Each package folder (`data/`, `model/`, `service/`, `web/`) must contain an empty `__init__.py` file.

4. Detailed Step-by-Step Instructions for Creating the Application

Step 1. Create the project and virtual environment

1. Create the folder `fastapi-image-processing`.
2. Open a terminal in this folder.
3. Create a virtual environment:

bash `python -m venv venv`

4. Activate it:

- Windows: `venv\Scripts\activate`
- macOS / Linux: `source venv/bin/activate`

Step 2. Install the libraries

Create the file `requirements.txt` and insert the following into it:

```
fastapi
uvicorn[standard]
python-multipart
pydantic
opencv-python-headless
numpy
```

Execute the command:

```
pip install -r requirements.txt
```

Step 3. Create the .env file

In the project root, create the file `.env`:

```
PICTURES_SQLITE_DB=db/images.db
```

Step 4. Create the Pydantic model (model/pictures.py)

```
# model/pictures.py
from pydantic import BaseModel, Field
import numpy as np
from datetime import datetime

class Picture(BaseModel):
    name: str
    img: np.ndarray
    description: str = ""
    dt: datetime = Field(default_factory=datetime.now)
```

```
    model_config = {
        "arbitrary_types_allowed": True  # required for np.ndarray
    }
```

Step 5. Configure the SQLite connection (data/init.py)

```
# data/init.py
"""Initialization of the SQLite database connection."""
import os
from pathlib import Path
from sqlite3 import connect, Connection, Cursor

conn: Connection | None = None
curs: Cursor | None = None

def get_db(name: str | None = None, reset: bool = False) -> None:
    """Connect to the database file."""
    global conn, curs
    if conn and not reset:
        return
    if not name:
        name = os.getenv("PICTURES_SQLITE_DB")
        if not name:
            top_dir = Path(__file__).resolve().parents[1]
            db_dir = top_dir / "db"
            db_dir.mkdir(exist_ok=True)
            name = str(db_dir / "images.db")
    conn = connect(name, check_same_thread=False)
    curs = conn.cursor()
    print(f" Connected to the database: {name}")

get_db()
```

Step 6. Implement CRUD operations (data/pictures.py)

```
# data/pictures.py
"""Data layer: working with images in SQLite (DB-API)."""

from .init import conn, curs
```

```
from model.pictures import Picture
import cv2
import numpy as np
from datetime import datetime
from error import Missing, Duplicate   # create the error.py file later

curs.execute('''
    CREATE TABLE IF NOT EXISTS pictures(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        description TEXT,
        image_data BLOB NOT NULL,
        created_at DATETIME DEFAULT CURRENT_TIMESTAMP
    )
''')
conn.commit()

def row_to_model(row: tuple) -> Picture | None:
    """Converts a row from the database into a Picture object (BLOB → np.ndarray)."""
    if row is None or len(row) < 5:
        return None
    try:
        _, name, description, image_bytes, created_at = row
        nparr = np.frombuffer(image_bytes, np.uint8)
        img = cv2.imdecode(nparr, cv2.IMREAD_COLOR)
        if img is None:
            return None
        dt = datetime.strptime(created_at, "%Y-%m-%d %H:%M:%S") if isinstance(created_at, s
        return Picture(name=name, img=img, description=description or "", dt=dt)
    except Exception:
        return None

def get_one(name: str) -> Picture | None:
    """Retrieve a single image by name."""
    qry = "SELECT * FROM pictures WHERE name = ?"
    curs.execute(qry, (name,))
```

```
    row = curs.fetchone()
    return row_to_model(row)

def add_one(picture: Picture) -> int:
    """Save a Picture object to the database (np.ndarray → BLOB)."""
    success, encoded_img = cv2.imencode('.png', picture.img)
    if not success:
        raise ValueError("Failed to encode the image")
    image_blob = encoded_img.tobytes()
    created_at_str = picture.dt.strftime('%Y-%m-%d %H:%M:%S')
    curs.execute('''
        INSERT INTO pictures (name, description, image_data, created_at)
        VALUES (?, ?, ?, ?)
    ''', (picture.name, picture.description, image_blob, created_at_str))
    conn.commit()
    return curs.lastrowid
```

Step 7. Add business logic (service/pictures.py)

```
# service/pictures.py
"""Service layer: business logic for working with images."""
from model.pictures import Picture
import data.pictures as pictures_data


def get_one(name: str) -> Picture | None:
    """
    Retrieve a single image by name.
    Returns a Picture object (with np.ndarray) or None.
    """
    return pictures_data.get_one(name=name)


def add_one(picture: Picture) -> int:
    """
    Save a Picture object to the database.
    Returns the ID of the new record.
```

```
    """
    return pictures_data.add_one(picture)
```

Step 8. Create FastAPI routers (web/pictures.py)

```
# web/pictures.py
from fastapi import APIRouter, UploadFile, File
from fastapi.responses import Response
import cv2
import numpy as np
from service.pictures import add_one, get_one
from model.pictures import Picture
from datetime import datetime

router = APIRouter(prefix="/pictures", tags=["pictures"])


@router.post("/", status_code=201)
async def upload_picture(file: UploadFile = File(...)):
    """Image upload."""
    contents = await file.read()
    nparr = np.frombuffer(contents, np.uint8)
    img = cv2.imdecode(nparr, cv2.IMREAD_COLOR)
    picture = Picture(
        name=file.filename,
        img=img,
        description="",
        dt=datetime.now()
    )
    picture_id = add_one(picture)
    return {"message": "Image successfully saved", "id": picture_id, "name": file.filename}


@router.get("/{name}")
def get_picture(name: str):
    """Retrieve an image by name."""
    picture = get_one(name)
```

```
    if picture is None:
        return {"error": "Image not found"}
    success, encoded_img = cv2.imencode('.png', picture.img)
    return Response(content=encoded_img.tobytes(), media_type="image/png")
```

Step 9. Create the main application file (src/main.py)

```
# src/main.py
from fastapi import FastAPI
from web.pictures import router as pictures_router

app = FastAPI(title="FastAPI Image Processing Lab")
app.include_router(pictures_router)

@app.get('/')
def top():
    return 'top here'

if __name__ == '__main__':
    import uvicorn
    uvicorn.run('main:app', reload=True)
```

Step 10. Run the application

```
uvicorn src.main:app --reload
```

Open in the browser: http://127.0.0.1:8000/docs

5. Basic SQLite Commands (DB-API) in the Context of Images

- `CREATE TABLE IF NOT EXISTS pictures(...)` — creates a table with the `image_data BLOB` column.
- `INSERT ... VALUES (?, ?, ?, ?)` — positional parameters.
- `SELECT * FROM pictures WHERE name = ?` — retrieves a record.
- `curs.fetchone()` + `row_to_model` — converts BLOB to `np.ndarray`.
- `conn.commit()` — commits changes.

6. How Images Are Saved and Retrieved

- **Saving:** `UploadFile.read()` → `bytes` → OpenCV (`imencode`) → `BLOB` → `INSERT` (via `service` → `data`).

- **Retrieving:** `SELECT image_data` → `bytes` → `cv2.imdecode` → `np.ndarray` → `Picture` object → `Response` with `media_type="image/png"`.

Practical Part of the Laboratory Work

Task for Independent Implementation

1. **Implement the image deletion operation**
   Add a function to the data layer that deletes a record by image name.
   Create the corresponding function at the service layer.
   At the web layer, add the HTTP DELETE method at the route `/pictures/{name}`, which accepts the image name and returns a deletion confirmation or an error message.
2. **Implement the image data modification (update) operation**
   Add the ability to update image metadata (for example, the `description` field).
   Implement a modification function at the data layer.
   Create a function at the service layer that accepts a new description and updates the record.
   At the web layer, add the HTTP PATCH (or PUT) method at the route `/pictures/{name}`, which accepts a new description and returns the updated object.
3. **Implement three image processing operations at the service layer**
   In the file `service/pictures.py` create three independent functions:
   - Convert an image to grayscale.
   - Edge detection (for example, using Canny).
   - Apply Gaussian blur.

   Each function must accept a `Picture` object, perform the corresponding processing using OpenCV, and return a new `Picture` object with the modified image (`img: np.ndarray`).
4. **Make the new operations available via the API**
   In the file `web/pictures.py` add three new endpoints (POST or GET) that allow each of the three processing operations to be called by image name.
   After processing, the image must be saved in the database under the same name (or under a new name if you decide to save a copy).
   The endpoints must return the updated image in PNG format.

Requirements

- All changes must strictly follow the three-tier architecture (data → service → web).
- Do not modify the existing upload and retrieval code.
- Handle possible errors (missing image, invalid data).
- Test all new functions via Swagger UI (`/docs`).
- Add comments to the new code in PEP 8 style.

What to Submit for Grading

1. The complete project code in an archive.
2. Screenshots of the new endpoints working in Swagger UI (DELETE, PATCH, three processing operations).
3. A short report (1–2 pages) describing the functions you added and how they work.

Control Questions for the Laboratory Work

1. What is a three-tier FastAPI application architecture and which folders correspond to the web, service, data, and model layers?
2. Why is an `__init__.py` file required in each package folder (`data/`, `model/`, `service/`, `web/`)?
3. Which environment variable sets the path to the SQLite database file in this project?
4. What fields does the Pydantic model `Picture` contain and why is `arbitrary_types_allowed: True` required for the `img` field?
5. How does the `row_to_model` function convert a BLOB from SQLite into a `numpy.ndarray` object?
6. Which SQL commands are used in the `add_one` function to insert an image into the `pictures` table?
7. Why is `cv2.imencode` used in the `add_one` function before saving to BLOB?
8. What is the role of the service layer (`service/pictures.py`) in the three-tier architecture?
9. Which HTTP method and route are used to upload a new image?
10. Which HTTP method and route are used to retrieve an image by name?
11. Which OpenCV method is used to convert an image to grayscale?
12. Which OpenCV method is used for edge detection?
13. Which OpenCV method is used to apply Gaussian blur?
14. Which HTTP method will you use to delete an image by name?
15. Which HTTP method will you use to update an image's description?
16. What happens in the `get_one` function if an image with the specified name is not found in the database?
17. Why is `conn.commit()` required after an `INSERT` in `data/pictures.py`?
18. Which file format is used when encoding an image in the processing functions (`imencode`)?
19. How will you organize calls to the three image processing operations (grayscale, edges, blur) via the API?
20. What advantages does dividing the code into three layers (web → service → data) provide when further developing the application (adding new AI algorithms)?